

A Retail Digital Rupiah Prototype on Hyperledger Fabric: Two-Tier Business Processes, Risk-Based KYC Tiers, and a Reproducible Benchmark

1st Muhammad Fachrizal Giffari

*Department of Electrical and
Information Engineering
Universitas Gadjah Mada*

Yogyakarta, Indonesia

muhammadfachrizalgiffari@mail.ugm.ac.id

2nd Teguh Bharata Adji

*Department of Electrical and
Information Engineering
Universitas Gadjah Mada*

Yogyakarta, Indonesia

adji@ugm.ac.id

3rd Guntur Dharma Putra

*Department of Electrical and
Information Engineering
Universitas Gadjah Mada*

Yogyakarta, Indonesia

gdputra@ugm.ac.id

Abstract—Bank Indonesia pursues Central Bank Digital Currency (CBDC) through Project Garuda, but the project’s publicly documented experiments cover only the wholesale tier. To our knowledge, no publicly available system provides a testable implementation of Bank Indonesia’s retail money business processes (issuance, two-tier distribution, risk-based customer onboarding, payment, redemption, and supervision) on a single auditable blockchain ledger. This paper presents a retail Digital Rupiah prototype deployed on a permissioned, five-organization Hyperledger Fabric network. Its Go chaincode anchors KYC status, risk level, due-diligence level, senior approval, expiry, and document hashes on the ledger, while raw identity data remains in an off-chain PostgreSQL store. The chaincode derives the BASIC, STANDARD, or MERCHANT wallet tier from an approved KYC profile and applies five per-tier transfer and balance limits, using the Fabric transaction timestamp so that every endorser obtains the same result. We formalize the tier-derivation function, transfer-validity predicate, and double-spend-safety argument, then evaluate the prototype with a reproducible Hyperledger Caliper protocol that varies client concurrency and exercises the business processes directly. Across transfer rounds at three concurrency levels and four business-process workloads, the prototype committed 814 of 816 transactions, and transfer throughput saturates at 14.50 ± 0.32 TPS regardless of client concurrency, with bounded latency (mean ≤ 0.99 s, maximum ≤ 2.63 s). Three dedicated workloads test the compliance and safety claims rather than inferring them: all 34 negative-path submissions across seven rule classes were rejected with no enforcement gap, every numeric cap held on both sides of its boundary, and concurrent transfers from a single wallet were serialized, with every losing transaction rejected at multi-version concurrency control validation. Container-level measurements place no component near saturation, so the ceiling is not explained by endorsement or database capacity. The prototype provides an auditable testbed for studying blockchain-based implementation of Bank Indonesia’s business processes, not a production-ready system.

Index Terms—Digital Rupiah, retail CBDC, Hyperledger Fabric, risk-based KYC, wallet limits, business process, blockchain benchmarking

I. INTRODUCTION

Central banks are evaluating Central Bank Digital Currency (CBDC) as a digital form of sovereign money [1].

Bank Indonesia pursues CBDC through Project Garuda. Its publicly documented proof of concept targets the *wholesale* tier, specifically a cash ledger for issuance and redemption among the central bank and large financial institutions [2], [3]. The Garuda White Paper also describes a retail tier in which intermediary banks and payment service providers (PJP) distribute Digital Rupiah to households and merchants [3]. That description remains at the policy and architecture level, and we are not aware of an openly available, testable system that implements Bank Indonesia’s retail money business processes end to end on a blockchain ledger.

Central-bank digital money involves a sequence of business processes, not a single ledger operation. The central bank *issues* money against reserves, banks *distribute* liquidity to PJP, and PJP *onboard* customers and merchants through risk-based Know-Your-Customer (KYC) checks. Holders then *transact* within category-specific limits, intermediaries *redeem* money to the central bank, and the supervisor *reconciles* and audits its circulation. A blockchain implementation must express these processes as deterministic transactions that each organization can validate independently. It must also preserve two-tier intermediation to reduce bank disintermediation [4] and give the supervisor an audit surface.

Most technical studies isolate one part of this problem: permissioned CBDC architecture [5], privacy [6], security threat models [7], or throughput in account-based systems [8]. Deployed retail systems, including Nigeria’s eNaira [9], China’s e-CNY [10], and the Bahamian Sand Dollar [11], publish design papers but not reproducible performance benchmarks. The surveyed literature therefore provides no integrated, openly measurable retail reference for the Indonesian two-tier model [12], [13].

To address this gap, we implement a Digital Rupiah prototype on a permissioned Hyperledger Fabric network [14]. The prototype maps Bank Indonesia’s business processes to chaincode transactions. This paper contributes:

- 1) A two-tier Digital Rupiah chaincode that integrates hierarchical issuance and distribution, risk-based KYC with

deterministic wallet-tier derivation, tiered transfer limits, and supervisor-facing reconciliation. This extends the wholesale-only Garuda proof of concept with a testable retail implementation.

- 2) A formal, deterministic, double-spend-safe transfer protocol that combines a tier-limit validity predicate, Fabric multi-version concurrency control (MVCC), and a transaction-timestamp clock, so that limit counters and validation are identical across endorsing organizations.
- 3) An open, reproducible retail-CBDC benchmark with Hyperledger Caliper [15] that documents the testbed, workload, population seeding, a client-concurrency sweep, and business-process profiles. The benchmark identifies the throughput saturation point, bounds the resource envelope at that point with container-level metrics, and pairs the contention-free load with negative-path, boundary-value, and single-key adversarial workloads that test policy enforcement, cap placement, and MVCC double-spend rejection as measured outcomes.

II. BACKGROUND AND RELATED WORK

A. Two-Tier Retail CBDC

In the two-tier (intermediated) model the central bank issues CBDC and remains the ultimate liability holder, while regulated intermediaries onboard end users, perform KYC, and operate customer-facing rails [1], [4]. This preserves the existing division of labor and limits the central bank’s exposure to retail operations. Bank Indonesia adopts this structure for Digital Rupiah, with banks and PJP acting as distributors between the central bank and the public [3].

B. Permissioned Blockchain and Hyperledger Fabric

Hyperledger Fabric is a modular, permissioned blockchain with pluggable ordering, channel-based data isolation, and separate execution, ordering, and validation stages [14]. Membership Service Providers (MSPs) govern identity, the crash-fault-tolerant Raft protocol orders transactions, and CouchDB can provide a queryable world state. Fabric has consequently been used in CBDC and regulated-finance prototypes [5], [16]. Hyperledger Caliper [15], [17] measures throughput and latency under different workloads.

C. Compliance, Limits, and Supervision

Retail CBDC policy calls for AML/CFT controls and holding limits to be part of the system design. In Indonesia, these controls are subject to anti-money-laundering regulation [18]. Token-based retail designs also face denial-of-service and double-spend risks [19], so policy enforcement must be deterministic. Our chaincode combines tiered limits, KYC gating, and supervision events, complementing studies that address privacy [6] or security [7] separately. Deployed retail CBDCs already document tiered, KYC-linked limits: e-CNY defines wallet categories with graduated balance and transaction ceilings [10], and eNaira applies tiered daily transaction and balance limits [9]. The closest published prototype is Project Aurum, a full-stack two-tier retail CBDC built for the Hong

TABLE I
COMPARISON WITH REPRESENTATIVE RETAIL CBDC SYSTEMS

System	Platform	2-tier	Limits	Open	Bench.
e-CNY [10]	Permissioned	✓	✓	×	×
eNaira [9]	Fabric	✓	✓	×	×
Sand Dollar [11]	Permissioned	✓	✓	×	×
OpenCBDC [8]	Account	×	×	✓	✓
This work	Fabric 2.5	✓	✓	✓	✓

Limits: tiered, KYC-linked holding or transaction limits described in public documentation. Open: implementation publicly available. Bench.: open, reproducible throughput/latency benchmark. OpenCBDC is account-based and not distributed-ledger-based. Rows span deployed national CBDCs and a research platform, and summarize public information rather than a controlled comparison.

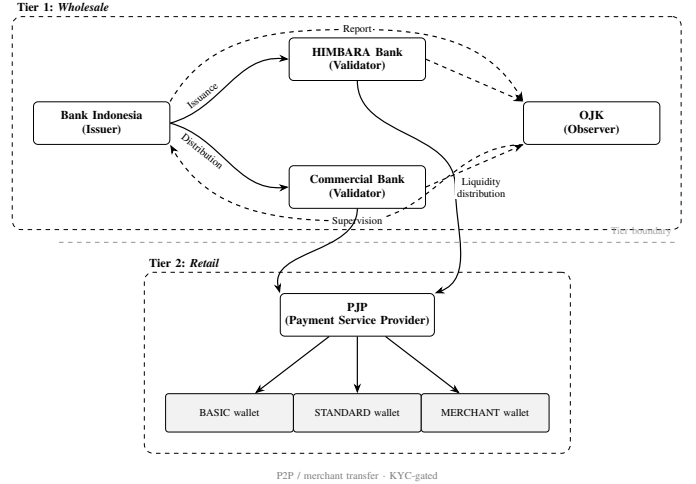


Fig. 1. Two-tier actor structure of the Digital Rupiah system. Bank Indonesia issues and distributes to validator banks (Tier 1); banks distribute liquidity to the PJP, which serves tiered retail wallets (Tier 2). The OJK observer receives reports and supervises.

Kong Monetary Authority [20], which pairs a wholesale inter-bank system with a retail e-wallet system and, like this work, keeps the two tiers architecturally distinct. Aurum reports no reproducible throughput benchmark, and its source code and technical manuals are released only to BIS member central banks rather than publicly. What distinguishes this prototype among the representative systems in Table I is that its two-tier intermediation and KYC-and-limit policy are realized in a publicly available implementation with a reproducible benchmark, rather than described in design documentation alone.

III. BANK INDONESIA BUSINESS PROCESSES

The system implements six processes in the circulation of central-bank digital money. Fig. 1 shows the actors in each tier. Table II identifies the initiator, chaincode transaction, and endorsement requirement for each process.

The wholesale tier contains Bank Indonesia, banks, and PJP. These institutional participants follow a lifecycle of submission, approval, freezing, unfreezing, rejection, and offboarding. Bank Indonesia mints Digital Rupiah to a treasury wallet. Banks request issuance and distribute liquidity to PJP; a

TABLE II
BANK INDONESIA BUSINESS PROCESSES AND THEIR CHAINCODE
REALIZATION

Process	Initiator	Chaincode transaction
Issuance	Bank Indonesia	Mint, RequestIssuance
Distribution	Validator bank	DistributeToParticipant
Onboarding/KYC	Bank / PJP	SubmitKycProfile, RefreshKycProfile, CreateWallet
Payment	Customer	Transfer
Redemption	Participant	Burn, RequestRedemption
Supervision	OJK / BI	GetReconciliationReport, GetMetrics, GetSupervisionEvents

All state-changing transactions commit under a MAJORITY endorsement policy (3 of 5 organizations).

TABLE III
NETWORK TOPOLOGY OF THE FIVE-ORGANIZATION FABRIC NETWORK

Organization	Role	Peer	CouchDB
BankIndonesiaOrg	Issuer	peer0.bi:7051	:5984
HimbaraBankOrg	Validator	peer0.himbara:9051	:6984
CommercialBankOrg	Validator	peer0.commercial:11051	:7984
PJPOrg	Distributor	peer0.pjp:13051	:9984
OJKObserverOrg	Supervisor	peer0.ojk:12051	:8984

Ordering: orderer.paynet:7050 (single-node Raft); per-org CA; TLS enabled

chaincode guard permits this validator-to-PJP flow but prevents a PJP from requesting issuance directly. The PJP serves customers and merchants in the retail tier. Institutional wallets have system-level limits but no retail tier.

IV. SYSTEM ARCHITECTURE

Fig. 2 shows the four-layer architecture: client, backend API, off-chain store, and Fabric network. The web client calls a Go REST backend built with Gorilla Mux. The backend authenticates requests with JWT and role-based access control, stores KYC and authentication data in PostgreSQL, and submits ledger transactions through the Fabric Gateway SDK. Fig. 3 details the permissioned network.

A. Network Topology

The network runs five peer organizations on one application channel (Fig. 3, Table III). *BankIndonesiaOrg* is the issuer; *HimbaraBankOrg* (HIMBARA, Indonesia’s association of state-owned banks) and *CommercialBankOrg* are validator and distributor banks; *PJPOrg* represents the payment service provider; and *OJKObserverOrg* is the supervisor (OJK, *Otoritas Jasa Keuangan*, the financial-services authority), which initiates no business transactions but whose peer still participates in endorsement under the majority policy. Each organization operates one peer with a CouchDB world state and maintains its own certificate authority and MSP for identity management. A single Raft orderer orders transactions; this configuration provides ordering but not crash fault tolerance, which requires at least three consenters. The MAJORITY endorsement policy requires at least three of the five organizations to validate a transaction before it can commit.

TABLE IV
CHAINCODE FUNCTION GROUPS

Group	Representative functions
Participants	SubmitParticipant, ApproveParticipant, Freeze/Unfreeze, Offboard
Wallets	CreateWallet, CreateWholesaleWallet, Freeze/UnfreezeWallet
Issuance	Mint, RequestIssuance, RequestIssuanceRtgs
Redemption	Burn, RequestRedemption
Distribution	DistributeToParticipant
Transfer	Transfer
KYC	SubmitKycProfile, RefreshKycProfile, GetKycProfile
Limits	SetTierLimit, SetSystemLimit
Supervision	GetReconciliationReport, GetMetrics, GetSupervisionEvents

TABLE V
BACKEND ROLES AND REPRESENTATIVE PERMISSIONS

Role	Representative permitted actions
Public	Health check, login
Authenticated	View own profile, list wallets
KYC-verified	Initiate transfer, view balance, redeem
Bank/PJP	Onboard customer, submit KYC, create wallet, distribute
Bank Indonesia	Issue, set system limits, approve participant
Merchant	Initiate transfer, view balance
Supervisor	Reconciliation, metrics, supervision events

B. On-Chain and Off-Chain Data

The Go chaincode in `digital_rupiah.go` groups functions for participants, wallets, issuance and redemption, retail transfers, KYC and compliance, limits, and supervision (Table IV). Its on-chain KYC anchor contains status, subject type, risk level, due-diligence level, senior-approval flag, expiry, and document hashes. Personally identifiable data remains in PostgreSQL. In the off-chain schema (Fig. 4), `kyc_profiles` links the provider-check and audit-event tables, while the separate `auth_users` table supports portal authentication.

A Go backend enforces role-based access control over seven roles, summarized in Table V, before any transaction reaches the chaincode.

The two layers enforce different things, and the split matters for how the prototype’s guarantees should be read. The chaincode enforces the monetary and compliance invariants of Section V: tier derivation, transfer limits, KYC validity, wallet freezing, system-level caps, and the validator-to-PJP issuance guard. Every endorsing peer re-evaluates them independently, so no single organization can commit a state change that violates them. Caller authorization is different: the mapping from roles to permitted functions in Table V is enforced only in the backend, and the chaincode does not additionally check the submitting client’s MSP identity against the requested function. A five-organization endorsement policy therefore protects the integrity of the rules but not the attribution of the request, and the single backend and PostgreSQL instance are a

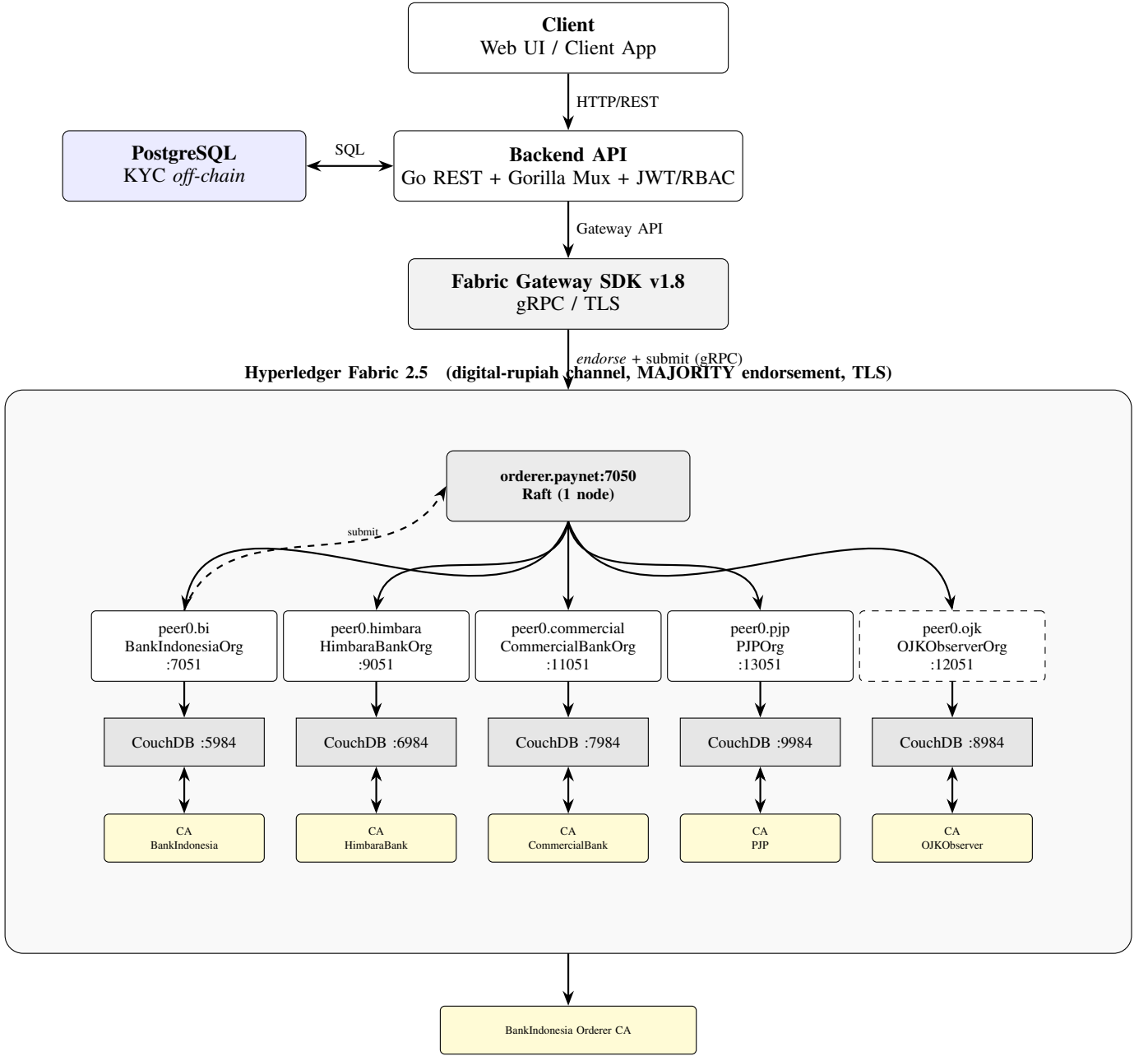


Fig. 2. Full system architecture of the Digital Rupiah prototype: client layer, Go REST backend with JWT/RBAC, off-chain PostgreSQL store, Fabric Gateway SDK, and the Hyperledger Fabric 2.5 network of five peer organizations with CouchDB, a per-organization CA, and a Raft orderer.

trust and availability concentration point. A production design would bind each organization to its own backend and re-check the submitter's certificate attributes inside the chaincode; Section VII returns to this.

V. PROTOCOL AND DETERMINISM

The chaincode applies the following compliance rules during endorsement. We formalize these rules and examine their behavior under concurrent transactions.

A. Wallet-Tier Derivation

Let s be the subject type, d the due-diligence level, and r the risk level of an approved KYC profile. The wallet tier τ is a deterministic function

$$\tau(s, d, r) = \begin{cases} \text{BASIC} & d=0, s=C, r=0 \\ \text{STANDARD} & d \geq 1, s=C, r < 2 \\ \text{MERCHANT} & d \geq 1, s=Mr, r \leq 2 \\ \perp & \text{otherwise,} \end{cases} \quad (1)$$

where $s \in \{C \text{ (customer)}, Mr \text{ (merchant)}\}$ is the subject type. The due-diligence level $d \in \{0, 1, 2\}$ denotes simplified,

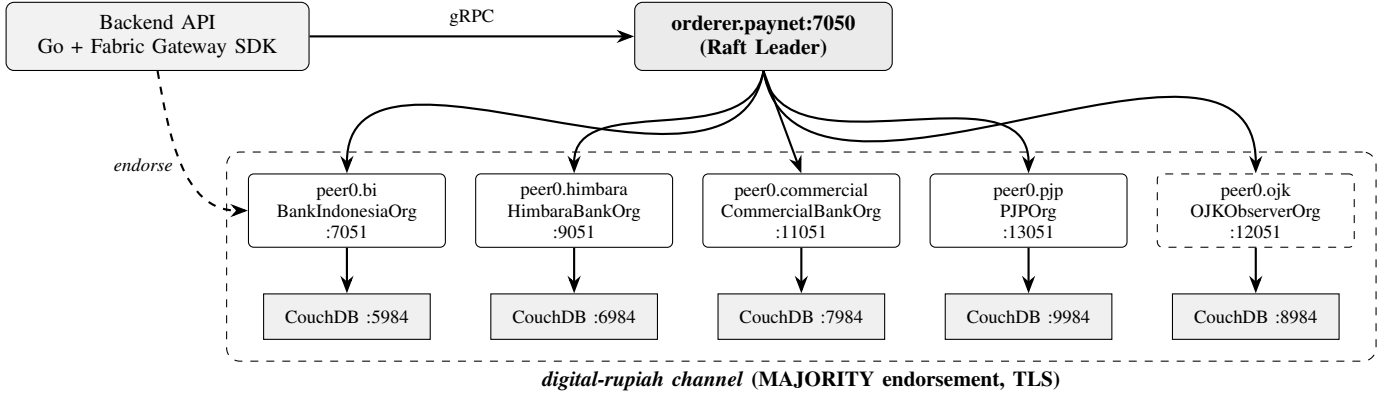


Fig. 3. Hyperledger Fabric network topology: five peer organizations, each with a CouchDB world state, on a single channel under a Raft orderer; the backend endorses via the Fabric Gateway SDK.

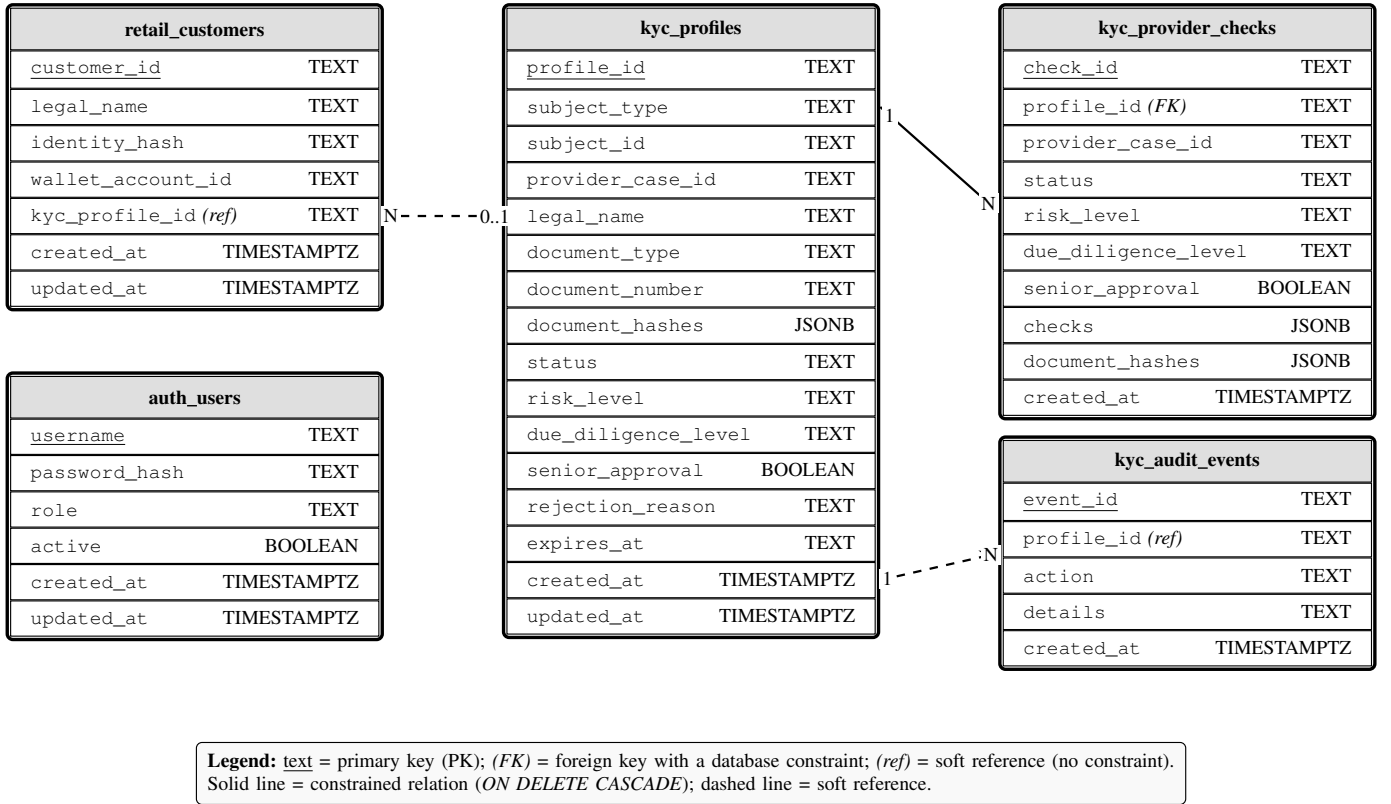


Fig. 4. Off-chain PostgreSQL entity-relationship schema. The `auth_users` table stands alone for portal authentication, while `kyc_profiles` is the hub linking the provider-check history and the audit trail. No raw identity data is written to the ledger.

standard, or enhanced due diligence, and the risk level $r \in \{0, 1, 2\}$ denotes low, medium, or high risk. Under standard or enhanced due diligence ($d \geq 1$), a customer maps to STANDARD and a merchant to MERCHANT at any of these three risk levels; simplified due diligence ($d=0$) is confined to a low-risk customer, who maps to BASIC. A high-risk subject ($r=2$) therefore remains tier-eligible, but only with enhanced due diligence and senior approval, both enforced when the KYC profile is approved. A fourth, prohibited risk classification is

rejected at approval and never reaches τ ; the value $\perp$ identifies any other profile that cannot support a retail wallet. Because τ uses only the approved profile, users cannot select their own tier.

B. Transfer-Validity Rules

Each wallet tier has five limits, listed in Table VI: a per-transaction cap, a daily outgoing cap, a monthly outgoing cap, a monthly incoming cap, and a maximum balance. For a transfer of amount a from a sender to a receiver, the chaincode

TABLE VI
RETAIL WALLET TIER LIMITS (INDONESIAN RUPIAH)

Limit	BASIC	STANDARD	MERCHANT
Max. balance	2,000,000	20,000,000	200,000,000
Per transaction	250,000	2,500,000	10,000,000
Daily outgoing	500,000	10,000,000	50,000,000
Monthly outgoing	5,000,000	40,000,000	500,000,000
Monthly incoming	20,000,000	40,000,000	500,000,000

TABLE VII
SYSTEM-LEVEL LIMITS (INSTITUTIONAL WALLETS)

Limit	Meaning	Value (Rp)
Total supply cap	Sum of all balances	10,000,000,000,000
Per-participant cap	Per-participant balance	1,000,000,000,000
Per-transaction cap	Per-transaction amount	100,000,000,000

admits the transfer only if every one of the following checks passes:

- neither the sender’s nor the receiver’s wallet is frozen;
- the sender’s KYC profile is approved and not expired;
- the amount does not exceed the sender’s per-transaction cap;
- the sender’s running daily and monthly outgoing totals, plus this transfer, stay within the sender’s daily and monthly outgoing caps;
- the receiver’s running monthly incoming total, plus this transfer, stays within the receiver’s monthly incoming cap;
- the sender has enough balance to cover the transfer; and
- the resulting receiver balance does not exceed the receiver’s maximum-balance cap.

If any check fails, the transfer is rejected. Institutional wallets (Bank Indonesia, banks, and PJP) skip these tier limits but are still bounded by the system-level caps in Table VII: a total supply cap on the sum of all balances, a per-participant balance cap, and a per-transaction cap.

The values in Table VI are chaincode parameters, seeded at ledger initialization and revisable through `SetTierLimit`, so limit policy is itself an auditable on-chain transaction rather than a compiled-in constant. The BASIC tier is anchored to existing Indonesian electronic-money regulation, which caps unregistered electronic money at Rp2,000,000 and caps monthly incoming transaction value at Rp20,000,000 [21]; both figures are reproduced exactly in the BASIC row, which lets the lowest tier stand in for the regulated no-KYC instrument. The STANDARD and MERCHANT tiers are not regulatory values. They are scaled above the electronic-money ceilings to represent bank-intermediated retail accounts, for which the same regulation sets no directly comparable limit, and they are test parameters rather than a calibration proposal: the merchant ceilings in particular sit well below the turnover of a mid-size Indonesian retailer. Setting defensible national values is a policy exercise outside the scope of this paper.

C. Determinism and Double-Spend Safety

Every endorsing peer evaluates the transfer rules independently. The daily/monthly counters and KYC-expiry check therefore cannot use a peer’s local clock, because clock differences could produce different decisions. Instead, the chaincode uses Fabric’s transaction timestamp, which is identical for all endorsers of a transaction, to select the applicable day and month windows. The running totals are not recomputed from transaction history. Each wallet record carries its own spent-today, spent-this-month, and received-this-month counters together with the day and month stamps at which they were last reset; a transfer derives the current day and month from the transaction timestamp, zeroes any counter whose window has rolled over, and then tests the limits against the updated values. Because every input to this evaluation is either the transaction timestamp or the two wallet records read during endorsement, each endorser computes the same totals and reaches the same decision.

Double-spend safety combines this deterministic check with Fabric’s built-in multi-version concurrency control (MVCC). An endorsement reads the current version of a wallet record, and Fabric commits the transaction only if that version remains unchanged through ordering and validation. Two concurrent transfers from the same wallet may endorse against the same record and version. Only the first ordered transaction commits. The second fails MVCC validation even if it passed the balance check during endorsement. Keeping the limit counters on the wallet record rather than in a separate index is what extends this protection to them: balance and counters share one key, so a single version check covers both, and no range or rich query enters the read set where a phantom read could arise. Fig. 5 summarizes the transfer routine.

VI. PERFORMANCE EVALUATION

A. Methodology

We measure performance with Hyperledger Caliper on Fabric 2.5.16, following blockchain performance-metric guidance [17]. Table VIII states the testbed. All eleven containers, the Caliper workers, and the backend share one host, so the figures below describe that configuration and are not portable to a distributed deployment. The ordering parameters are listed because they bound the results directly: with a two-second batch timeout and a ten-message maximum, a block is cut whenever either condition is met first. Before measurement, the ledger is seeded with the population in Table X: 200 retail customers (20% BASIC, 80% STANDARD) and 50 merchants. Every account holds an approved KYC anchor and receives funding within its tier limits.

The experiment uses four workload families. The transfer workload comprises 70% customer-to-customer transfers, 25% customer-to-merchant transfers, and 5% reads (Table IX). It runs at three client-concurrency levels (1, 2, and 4 Caliper workers), with warmup, sustained, and peak rounds at each level, and is then repeated for five identical peak rounds so that throughput can be reported with a confidence interval. Every

```

1: function TRANSFER(sender, receiver, amount)
2: timestamp  $\leftarrow$  GETTXTIMESTAMP() {same value on every endorser}
3: if sender frozen or receiver frozen or sender's KYC not valid at timestamp then
4:   return REJECT
5: end if
6: derive day and month from timestamp; zero any sender or receiver counter whose window has rolled over
7: dailyOut, monthlyOut  $\leftarrow$  sender's stored outgoing counters
8: monthlyIn  $\leftarrow$  receiver's stored incoming counter
9: if any transfer-validity rule fails (Sec. V) then
10:  return REJECT
11: end if
12: sender.balance  $\leftarrow$  sender.balance - amount; increment sender's daily and monthly outgoing counters
13: receiver.balance  $\leftarrow$  receiver.balance + amount; increment receiver's monthly incoming counter
14: append record to sender's and receiver's history; emit supervision event if high-risk
15: PutState (validated by Fabric MVCC)
16: return COMMIT

```

Fig. 5. Deterministic, double-spend-safe transfer executed on every endorsing peer. “Sender’s KYC not valid” means not approved or expired; “transfer-validity rule” refers to the checklist in Sec. V.

TABLE VIII
TESTBED CONFIGURATION

Component	Specification
Processor	Intel Core i7-7700HQ @ 2.80 GHz (4 cores, 8 threads)
Memory	16 GB
Host OS	Fedora Linux 44, kernel 7.1.3
Runtime	Docker 29.6.2
Fabric	2.5.16 (peer, orderer); CouchDB 3.2
Chaincode	Go, fabric-contract-api-go v2.0.0
Backend	Go 1.23, fabric-gateway v1.8.0
Benchmark	Hyperledger Caliper 0.6.0
Ordering	Single-node Raft; BatchTimeout 2 s; MaxMessageCount 10; PreferredMaxBytes 2 MB; AbsoluteMaxBytes 10 MB
Endorsement	MAJORITY (3 of 5 organizations)

round submits 40 transactions, which at the achieved rates lasts roughly three to six seconds; rounds this short capture the saturated steady state only briefly, which is why the peak round is the one repeated. Intervals reported as $\pm$ are two-sided 95% Student’s t intervals over those five rounds ($n=5$, four degrees of freedom). Four functionality profiles exercise the business processes in Section III: KYC onboarding, monetary operations (issuance, distribution, redemption), supervision reads, and admin/policy.

Three further workloads test the compliance and safety claims directly. The conformance workload submits only transactions that each violate one policy rule, so enforcement

TABLE IX
CALIPER TRANSFER WORKLOAD COMPOSITION

Share	Operation	Transaction
70%	Customer-to-customer transfer	Transfer
25%	Customer-to-merchant transfer	Transfer
5%	Wallet read	GetWallet

is observed as an outcome rather than inferred from an absence of rejections. The boundary workload then probes each numeric cap from both sides, pairing a submission exactly at the limit with one a single rupiah past it, which distinguishes a correctly placed cap from an off-by-one comparison. The adversarial workload directs four concurrent workers at a single sender wallet, forcing repeated MVCC version conflicts on one balance key. The transfer and adversarial designs are deliberately separate: the contention-free transfer workload measures the throughput ceiling without abort noise, while the single-key workload measures conflict handling without confounding it with saturation.

The conformance workload needs a metric of its own, because for it a rejection is the desired result and Caliper scores every rejected submission as a failed transaction. The workload therefore keeps its own counters: a submission that the chaincode rejects is recorded as *enforced*, and one that commits is recorded as an enforcement *gap* and re-raised so that it cannot pass silently. The conformance verdict is the gap count, not Caliper’s success ratio. The round is configured for 42 transactions, of which 34 are policy-violating submissions and the remainder are fixture-seeding calls that are expected to succeed; we report the 34 assertions and their verdict.

During every round, Caliper’s Docker resource monitor samples processor, memory, network, and disk use once per second for all eleven containers, comprising five peers, five CouchDB instances, and the orderer. Processor figures are reported with Caliper’s CPU normalization enabled, which divides each container’s share of system-wide CPU time by the number of logical cores. A reported value is therefore a percentage of the whole host’s eight-thread capacity, not of a single core: 100% would mean the container saturated the entire machine.

For each round, we report throughput and efficiency η . Throughput is the number of successful transactions divided by wall-clock duration, while efficiency divides throughput by the offered (requested) transaction rate. A value near 1 indicates that the system keeps pace with the offered load. A value well below 1 indicates saturation. All reported figures come from the Caliper report generated against the deployed chaincode on a clean ledger.

B. Results

The performance workloads committed 814 of 816 transactions. The two failures occurred in the four-worker peak round, where a seeded sender exhausted its balance; this is an artifact of workload seeding rather than a system fault.

TABLE X
SEEDED TEST POPULATION AND WORKLOADS

Parameter	Value
Retail customers	200 (20% BASIC, 80% STANDARD)
Merchants	50 (MERCHANT tier)
Client concurrency	1, 2, 4 Caliper workers
Transfer rounds	warmup / sustained / peak
Repetition rounds	5 identical peak rounds
Functionality profiles	KYC, monetary, supervision, admin
Performance transactions	816
Conformance / boundary	34 / 8
Adversarial	200

Conformance counts the policy-violating submissions asserted, not the Caliper round size (42), which also contains fixture-seeding calls. Boundary counts the at-limit and one-past-limit assertion pairs.

TABLE XI
TRANSFER THROUGHPUT SCALING BY CLIENT CONCURRENCY

Wk.	Round	Send (TPS)	Thr. (TPS)	η	Avg (s)	Max (s)
1	Warmup	10.3	7.7	0.75	0.81	2.19
1	Sustained	30.8	12.7	0.41	0.64	2.15
1	Peak	58.7	14.7	0.25	0.64	2.22
2	Warmup	10.5	7.8	0.74	0.77	2.12
2	Sustained	31.6	12.5	0.40	0.63	2.21
2	Peak	62.9	14.8	0.24	0.66	2.27
4	Warmup	11.1	8.1	0.73	0.82	2.13
4	Sustained	31.9	11.1	0.35	0.99	2.63
4	Peak	66.6	14.3	0.21	0.62	2.32

All rounds: 40 transactions. Failures: 0 in every round except the four-worker peak round (2, insufficient balance). Wk.: Caliper workers.

The predicate in Section V produced consistent endorsement decisions throughout the run.

Table XI reports transfer scaling. Peak throughput saturates near 14.5 TPS and does not increase when client concurrency rises from one to four workers, which excludes a client-side limit. Efficiency falls from $\eta \approx 0.75$ during warmup to $\eta \approx 0.21$ at peak, placing the saturation point below the sustained offered load. Latency remains bounded: the mean is at most 0.99 s and the maximum is at most 2.63 s. Across five identical peak rounds, throughput is 14.50 TPS with a 95% confidence interval of ± 0.32 TPS (standard deviation 0.26 TPS), mean latency 0.68 ± 0.04 s, and maximum latency 2.31 ± 0.09 s, so the saturation point is stable across repetitions. Efficiency is already below one in the warmup rounds, at an offered rate near 10 TPS: these rounds start against a cold chaincode container and empty peer caches, and at 40 transactions they are too short for that startup cost to amortize. Fig. 6(a) shows the resulting curve: the three concurrency levels trace the same trajectory, which departs from the ideal $\eta=1$ line as offered load grows and then flattens.

Table XII reports the four business-process profiles. The short, single-key writes used by monetary operations produce the highest throughput (19.9 TPS) and lowest mean latency (0.32 s). Supervision reads sustain 16.3 TPS, with 1.17 s mean and 3.43 s maximum latency; reconciliation and metrics scan the whole transaction index through composite-key range

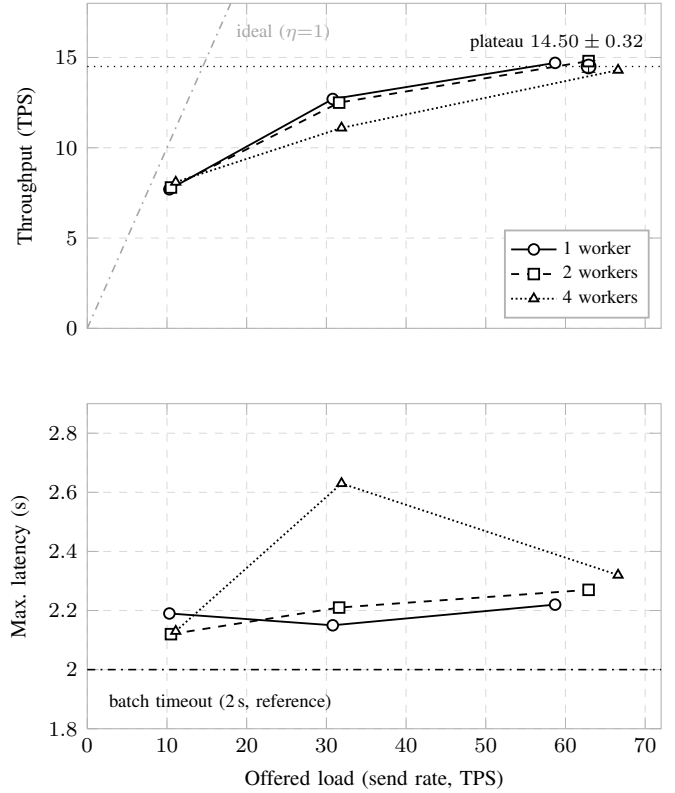


Fig. 6. Transfer performance against offered load at three client-concurrency levels. (a) Achieved throughput departs from the ideal $\eta=1$ line and flattens at 14.50 ± 0.32 TPS (95% confidence interval over five repeated peak rounds), and the three concurrency levels coincide, so additional client workers do not raise the ceiling. (b) Maximum latency stays between 2.12 and 2.63 s in every round. The two-second batch timeout is drawn for reference, but at 14.5 TPS a ten-message block fills before that timeout expires; the tail is consistent with queueing while the fixed-rate round drains, not with timeout-driven block cutting.

TABLE XII
BUSINESS-PROCESS FUNCTIONALITY PROFILES

Profile	Tx	Send (TPS)	Thr. (TPS)	Avg (s)	Max (s)
KYC onboarding	56	4.4	3.8	0.88	2.26
Monetary ops	80	20.5	19.9	0.32	0.56
Supervision reads	80	30.7	16.3	1.17	3.43
Admin / policy	40	5.3	5.2	0.92	1.72

queries rather than reading a maintained aggregate, so their cost grows with ledger size. The multi-step KYC onboarding and admin/policy profiles reach 3.8 TPS and 5.2 TPS, respectively. Both complete without failure, and the corresponding business processes execute on the ledger.

C. Policy Enforcement and Contention

Throughput results alone cannot show that the compliance rules reject anything, because a workload built from valid transactions can never trigger them. The conformance workload therefore submits only violations. Across 34 assertions spanning the seven rule classes in Table XIII, split evenly

TABLE XIII
NEGATIVE-PATH CONFORMANCE RESULTS

Rule violated by the submission	Expected	Observed
Per-transaction cap	reject	reject
Sender balance insufficient	reject	reject
Receiver maximum balance	reject	reject
Sender wallet frozen	reject	reject
Sender KYC anchor expired	reject	reject
Prohibited-risk approval	reject	reject
High risk without senior approval	reject	reject

34 policy-violating submissions across the seven classes; 0 committed, 0 enforcement gaps.

between two workers, every submission was rejected and the run recorded no enforcement gap.

Rejecting an input that sits well inside the violating region does not show that a cap is placed correctly, so a second workload probes each numeric limit from both sides: one transfer whose value lands exactly on the limit, which must commit, and one a single rupiah past it, which must be rejected. A rule with an off-by-one comparison passes the first workload and fails this one. Caliper issues submissions concurrently, so each of the eight assertions uses its own sender and receiver, and every accumulated state, whether a partial daily spend, a receiver pre-filled to its ceiling, or a drained sender, is established during seeding; no assertion can therefore be rejected by an MVCC conflict rather than by the rule under test. All eight outcomes matched (Table XIV). The per-transaction, daily-outgoing, and receiver-balance caps and the sender-balance check are inclusive and placed exactly where Section V specifies.

The adversarial workload then tests double-spend safety directly. Four workers issued 200 concurrent transfers from one sender wallet, so every transaction read the same balance version. Twelve committed and 188 were rejected at validation, and the twelve that committed were serialized rather than applied against a stale balance. This is the behavior argued in Section V: transactions that pass the balance check during endorsement still fail MVCC validation once an earlier transaction has advanced the version of that key. The argument predicts the serialization, not the split between the two outcomes; how many of the 200 commit depends on the offered rate and on endorsement latency, so the 94% rejection share characterizes this run rather than the protocol. The zero conflicts recorded by the contention-free transfer workload and the 188 recorded here are consistent, because the two workloads differ only in whether they share a sender key.

D. Discussion

These measurements characterize a single-host functional prototype, not national capacity. Transfer throughput plateaus near 14.5 TPS and monetary operations near 20 TPS. Both are modest for permissioned DLT and, like other private-DLT retail prototypes, remain far below the throughput of centralized card networks.

TABLE XIV
BOUNDARY-VALUE CONFORMANCE RESULTS

Limit probed	Exactly at limit	One past limit
Per-transaction cap (Rp250,000)	commit	reject
Daily outgoing cap (Rp500,000)	commit	reject
Receiver max. balance (Rp2,000,000)	commit	reject
Sender balance (exact drain)	commit	reject

BASIC-tier limits. Observed outcome matched the expected outcome in all eight assertions, so every cap is inclusive. Each assertion used a dedicated sender and receiver.

Throughput remains almost unchanged from one to four client workers, which already excludes a client-side limit, and the resource measurements narrow the alternatives further. At the highest offered load no monitored container comes close to saturation. Expressed as a share of the whole host, mean processor use stays at or below 0.08% on the peers, 0.13% on the CouchDB instances, and 0.01% on the orderer, with per-sample peaks of 0.20%, 0.34%, and 0.03% respectively; peer memory remains flat between 123 and 127 MB, and peer disk writes stay near one megabyte per round. No component is close to exhausting processor, memory, or write bandwidth, so the ceiling is not a capacity limit in any single container.

Locating it more precisely is harder than the aggregate figures suggest, and two candidate mechanisms remain. Sustained throughput of 14.5 TPS against a ten-message block maximum corresponds to a block cycle of roughly 0.7 s. That cycle is consistent both with the ordering service cutting and delivering blocks at that interval and with the serialized per-channel validation and commit path taking that long to apply a ten-transaction block to five peers and their CouchDB instances. Low aggregate processor use does not discriminate between them, because a commit path bounded by write latency and by sequential validation can throttle throughput while leaving every container mostly idle, which is the regime other Fabric performance studies also report [22], [23].

The latency data does not settle the question either, and one apparent clue should be discounted. Maximum latency sits between 2.12 and 2.63 s in every round, close enough to the two-second batch timeout to invite the reading that block cutting is timeout-driven. That reading does not survive the arithmetic: at 14.5 TPS a ten-message block fills in about 0.7 s, so the timeout is not the binding condition at peak load. The latency tail is better explained by queueing in the fixed-rate round. A peak round offers its 40 transactions in roughly 0.6 s and the system drains them at 14.5 TPS over about 2.8 s, which places the last transaction's latency near 2.2 s, as observed. Distinguishing block formation from commit serialization therefore requires an experiment this study did not run: a sweep of `MaxMessageCount` and `BatchTimeout`, with block-interval timestamps read back from the ledger. We report the plateau and its resource envelope as measured, and leave the attribution open.

Enforcement and safety rest on direct observation rather than on the absence of counterexamples. All 34 conformance submissions were rejected with no enforcement gap, every

numeric cap held on both sides of its boundary, and the concurrent single-key transfers were serialized, every losing transaction failing at MVCC validation. These outcomes are what the determinism and double-spend arguments in Section V require. The boundary result is the sharper of the two enforcement checks, since an off-by-one cap would leave the negative-path run looking identical.

Absolute TPS values should not be compared directly across platforms because the platform, measurement tool, network size, and workload differ among studies. Instead, the documented protocol fixes the workload mix, seeded population, concurrency sweep, and business-process profiles so that the saturation point can be measured again under another configuration.

VII. LIMITATIONS AND FUTURE WORK

The evaluation uses a single host with one peer per organization and no geographic distribution or network latency. Its Caliper results therefore apply only to the reported configuration and do not estimate national-scale performance. The workload pair also brackets rather than samples realistic traffic: the transfer workload holds every sender key disjoint, while the adversarial workload drives all writers onto one key. Retail payment traffic lies between these extremes, since popular merchants attract skewed but not exclusive concurrency. A graded-skew workload would be needed to measure throughput under partial contention, and the throughput figures reported here should be read as the contention-free ceiling.

Two caveats bound what the measurements support. The conformance and boundary workloads cover the numeric caps and the seven rule classes, but not the temporal edges: expiry exactly at the transaction timestamp and the rollover of a daily or monthly window are untested, and both depend on timestamp handling rather than on an integer comparison. And while the resource envelope rules out capacity exhaustion as the cause of the throughput plateau, it does not separate block formation from validation and commit serialization; that attribution awaits the parameter sweep described below.

Authorization is the prototype's clearest architectural limitation. As Section V describes, the chaincode enforces the monetary and compliance invariants at every endorser, but the mapping from caller roles to permitted functions lives only in the backend, which does not re-present the submitter's identity for chaincode-side checking. The five-organization endorsement policy consequently secures what may be written, not who asked for it, and a single backend and database concentrate trust that the network topology otherwise distributes. Binding each organization to its own backend and enforcing role checks from client certificate attributes inside the chaincode is the natural next step, and it would let a supervisor verify request provenance from the ledger alone. Related is the position of the OJK peer, which endorses every transaction it will later audit; excluding the supervisor from the endorsement policy while retaining full ledger visibility is the alternative worth evaluating. The single orderer is likewise a functional convenience rather than a design: it provides ordering without

crash fault tolerance, so its loss halts settlement until it is restarted.

The KYC process is programmatic and has no connection to a real eKYC provider. Authorized peers can see transaction data because the ledger uses neither private data collections nor privacy-preserving cryptography, which leaves the tension between supervisory visibility and holder privacy unaddressed rather than resolved. The prototype also omits payment-queue gridlock resolution and offline, network-disconnected payment. Offline payment would require secure devices and post-reconnection settlement of the kind studied in Project Polaris [24]. Connections to domestic infrastructure, including BI-RTGS and BI-FAST, remain stubs. The prototype thus demonstrates internal integration but not the full interoperability and interconnection required for Digital Rupiah.

Future evaluation should use distributed multi-host deployments, a graded-skew contention workload, and a sweep of the block-cutting parameters combined with block-interval and commit-phase instrumentation, which is what would separate the two candidate explanations for the current ceiling. Further implementation work should integrate real eKYC and identity controls, move caller authorization into the chaincode, test privacy-preserving supervision, and add gridlock resolution and offline payment.

VIII. CONCLUSION

This paper implements Bank Indonesia's digital money business processes in a two-tier Digital Rupiah prototype on a five-organization Hyperledger Fabric network. One chaincode integrates hierarchical issuance and distribution, risk-based KYC with deterministic wallet-tier derivation, tiered transfer limits based on the transaction timestamp, and supervisor-facing reconciliation.

The formal model specifies the tier-derivation function, transfer-validity predicate, and double-spend-safety argument. Under the reproducible Hyperledger Caliper protocol, the prototype committed 814 of 816 transactions across a client-concurrency sweep and four business-process workloads, and transfer throughput saturated at 14.50 ± 0.32 TPS with bounded latency. Three further workloads test the claims directly: all 34 negative-path submissions across seven rule classes were rejected with no enforcement gap, every numeric cap held on both sides of its boundary, and concurrent transfers from a single wallet were serialized, every losing transaction failing at MVCC validation, as the double-spend argument requires. Resource measurements show no container near saturation at the plateau, so the ceiling is not a capacity limit; separating block formation from commit serialization as its cause requires a block-parameter sweep that we leave to future work.

The prototype extends the public, wholesale-only Project Garuda experiments with an auditable and measurable retail reference. It provides a testbed for studying blockchain-based central-bank business processes but does not establish production readiness.

ARTIFACT AVAILABILITY

The chaincode, the five-organization network definition, the Go backend, and the Hyperledger Caliper workloads and benchmark configurations used in this paper are available at <https://github.com/mfachrizalg/bi-coin>.

REFERENCES

- [1] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. X. Zhou, “A survey of research on retail central bank digital currency,” *IMF Working Papers*, 2020.
- [2] Bank Indonesia, “Proyek garuda: Wholesale digital rupiah cash ledger — proof of concept report,” Bank Indonesia, Jakarta, Tech. Rep., 2024.
- [3] —, “Proyek garuda: Menavigasi arsitektur digital rupiah — white paper central bank digital currency,” Bank Indonesia, Jakarta, Tech. Rep., 2022.
- [4] R. A. Auer and R. Böhme, “The technology of retail central bank digital currency,” *BIS Quarterly Review*, pp. 85–100, Mar. 2020. [Online]. Available: https://www.bis.org/publ/qrpdf/r_qt2003j.htm
- [5] Bhawana and S. Kumar, “Permission blockchain network based central bank digital currency,” in *2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON)*. IEEE, 2021, pp. 1–6.
- [6] M. M. Islam and H. P. In, “A privacy-preserving transparent central bank digital currency system based on consortium blockchain and unspent transaction outputs,” *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2372–2386, 2023.
- [7] J. Hans, S. Khan, and D. Svetinovic, “Blockchain cbdc security threats using stride,” in *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2023, pp. 522–529.
- [8] J. Lovejoy, C. Fields, M. Virza, T. Frederick, D. Urness, K. Karwaski, A. Brownworth, and N. Narula, “A high performance payment processing system designed for central bank digital currencies,” *IACR Cryptology ePrint Archive*, vol. 2022, p. 163, 2022. [Online]. Available: <https://eprint.iacr.org/2022/163>
- [9] J. Ree, “Nigeria’s enaira, one year after,” International Monetary Fund, IMF Working Paper WP/23/104, 2023.
- [10] People’s Bank of China, “Progress of research and development of e-cny in china,” People’s Bank of China, Tech. Rep., 2021. [Online]. Available: <https://www.pbc.gov.cn/en/3688110/3688172/4157443/4293696/2021072014364791207.pdf>
- [11] Central Bank of The Bahamas, “Project sand dollar: A digital fiat currency for the bahamas,” Central Bank of The Bahamas, Nassau, Tech. Rep., 2020. [Online]. Available: <https://www.centralbankbahamas.com/viewPDF/documents/2019-12-25-02-18-11-Project-Sanddollar.pdf>
- [12] V. Sethaput and S. Innet, “Sok: Blockchain applications in central bank digital currencies (cbdcs),” in *2025 7th International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2025, pp. 195–202.
- [13] J. N. Aryani, “Designing central bank digital currency in indonesia: A critical literature review,” 2023, tugas Akhir Sarjana, Sekolah Bisnis dan Manajemen, Institut Teknologi Bandung.
- [14] E. Androuraki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, “Hyperledger fabric: A distributed operating system for permissioned blockchains,” in *Proceedings of the Thirteenth EuroSys Conference (EuroSys ’18)*. Porto, Portugal: ACM, 2018, pp. 1–15.
- [15] Hyperledger Foundation, “Hyperledger caliper: A blockchain performance benchmark framework,” 2024, accessed: Jun. 27, 2026. [Online]. Available: <https://hyperledger.github.io/caliper/>
- [16] L. I. Hanif, “Rancang bangun sistem blockchain aplikasi meetcoin menggunakan kerangka kerja hyperledger fabric,” Universitas Gadjah Mada, Yogyakarta, Skripsi Sarjana, Departemen Teknik Elektro dan Teknologi Informasi, 2024.
- [17] Hyperledger Performance and Scale Working Group, “Hyperledger blockchain performance metrics,” Hyperledger Foundation, Tech. Rep., 2018.
- [18] Bank Indonesia, “Peraturan bank indonesia nomor 10 tahun 2024 tentang penerapan anti pencucian uang, pencegahan pendanaan terorisme, dan pencegahan pendanaan proliferasi senjata pemusnah massal bagi pihak yang diatur dan diawasi oleh bank indonesia,” 2024. [Online]. Available: https://www.bi.go.id/id/publikasi/peraturan/Pages/PBI_102024.aspx
- [19] A. Ahuja, S. Pani, S. Kanchanapalli, V. R. R. M. A., and S. Lodha, “Limbocoin: On the denial-of-service of token based retail cbdcs,” in *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2024.
- [20] Bank for International Settlements, “Project aurum: A prototype for two-tier central bank digital currency (cbdc),” BIS Innovation Hub Hong Kong Centre, Hong Kong Monetary Authority and Hong Kong Applied Science and Technology Research Institute, Tech. Rep., Oct. 2022. [Online]. Available: <https://www.bis.org/publ/othp57.pdf>
- [21] Bank Indonesia, “Peraturan bank indonesia nomor 20/6/pbi/2018 tentang uang elektronik,” 2018. [Online]. Available: <https://www.bi.go.id/id/publikasi/peraturan/Pages/PBI-200618.aspx>
- [22] M. Kuzlu, M. Pipattanasomporn, L. Gurses, and S. Rahman, “Performance analysis of a hyperledger fabric blockchain framework: Throughput, latency and scalability,” in *2019 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 2019, pp. 536–540.
- [23] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, “BLOCKBENCH: A framework for analyzing private blockchains,” in *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM, 2017, pp. 1085–1100.
- [24] Bank for International Settlements, “Project polaris part 1: A handbook for offline payments with cbdc,” BIS Innovation Hub, Tech. Rep., 2023.